

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

SENSOR BERDASARKAN BASIS PENGUKURAN & AKTUATOR MOTOR

Anak Agung Ngurah Agung Kresna Ananta - 5024241085

2026

1 Langkah-Langkah Percobaan

1.1 Persiapan

Modul 2 memakai ESP32 DevKit v1 dengan framework Arduino, sama seperti Percobaan 3–5 pada Modul 1. Praktikan membuat project PlatformIO baru bernama `modul2-sensor-aktuator` dengan board Espressif ESP32 Dev Module dan framework Arduino. File `platformio.ini` berisi environment `esp32dev` dengan `platform = espressif32`, `framework = arduino`, dan `monitor_speed = 115200`. Untuk Percobaan 5, library ESP32Servo ditambahkan lewat barisW lib_deps = madhephaestus/ESP32Servo@^3.0.0. Setiap percobaan memakai rangkaian sendiri, jadi sebelum merakit percobaan berikutnya praktikan mematikan catu daya motor atau ESC, melepas rangkaian sebelumnya dari breadboard dan GPIO, lalu memasang hanya komponen yang dibutuhkan. Alur kerja di setiap percobaan sama: kode program ditulis di `src/main.cpp`, di-Build dan di-Upload, lalu hasilnya diamati lewat Serial Monitor (115200 baud). Isi `platformio.ini` juga tidak berubah, kecuali pada Percobaan 5 dan 6.

1.2 Percobaan 1: Sensor Resistif, Kapasitif, dan Induktif

Percobaan ini bertujuan membaca tiga jenis sensor dengan basis pengukuran yang berbeda: joystick KY-023 (resistif), modul touch TTP223 (kapasitif), dan sensor hall effect (induktif). Pada joystick, VRx dihubungkan ke GPIO 34, VRy ke GPIO 35, dan SW ke GPIO 27. Kedua pin VRx dan VRy berada di ADC1. Pin OUT modul TTP223 dihubungkan ke GPIO 4, dan pin DO sensor hall effect ke GPIO 26. Ketiga modul mendapat daya dari pin 3.3V dan GND ESP32.

Setelah Kode Program 1 di-Upload, praktikan menggerakkan dan menekan joystick, menyentuh pad TTP223, lalu mendekatkan magnet ke sensor hall effect sambil membaca perubahan nilainya di Serial Monitor.

Listing 1: Pembacaan joystick KY-023, TTP223, dan hall effect

```
#include <Arduino.h>

#define JOY_VRX 34
#define JOY_VRY 35
#define JOY_SW  27
#define TOUCH_PIN 4  // OUT modul TTP223
#define HALL_PIN 26

void setup() {
  Serial.begin(115200);
  pinMode(JOY_SW, INPUT_PULLUP); // SW terhubung ke GND saat ditekan
  pinMode(TOUCH_PIN, INPUT);
  pinMode(HALL_PIN, INPUT);
}

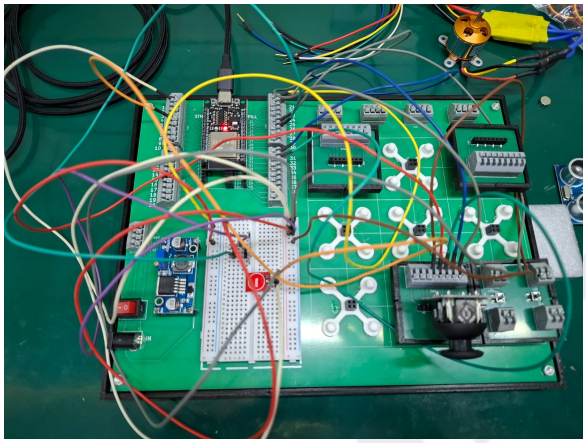
void loop() {
  int vrx = analogRead(JOY_VRX);
  int vry = analogRead(JOY_VRY);
  bool pressed = (digitalRead(JOY_SW) == LOW); // aktif LOW (internal pull-up)

  bool touched = (digitalRead(TOUCH_PIN) == HIGH); // TTP223 umumnya aktif HIGH
```

```
  bool magnetDetected = (digitalRead(HALL_PIN) == LOW); // umumnya aktif LOW, cek
    datasheet modul

  Serial.printf("Joystick VRx: %d | VRy: %d | SW: %s | Touch (TTP223): %s | Hall
    Effect: %s\n",
                vrx, vry,
                pressed ? "DITEKAN" : "idle",
                touched ? "TERSENTUH" : "idle",
                magnetDetected ? "MAGNET TERDETEKSI" : "idle");
  delay(300);
}
```

Tabel 1: Hasil Percobaan 1



(a) Rangkaian joystick KY-023, TTP223, dan hall effect

```
Executing task: C:\Users\raina\.platformio\penv\Scripts\platformio.exe device monitor
Joystick VRx: 1456 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1456 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1489 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1495 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1559 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1568 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1573 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 1606 | VRy: 0 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
```

(b) Serial Monitor saat joystick digerakkan

```
Problems  Output  Debug Console  Terminal  Ports
Executing task: C:\Users\raina\.platformio\penv\Scripts\platformio.exe device monitor
Joystick VRx: 2796 | VRy: 2829 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 2787 | VRy: 2826 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 2789 | VRy: 2823 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 2786 | VRy: 2819 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 2786 | VRy: 2821 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 2789 | VRy: 2823 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 2786 | VRy: 2816 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
Joystick VRx: 2789 | VRy: 2819 | SW: idle | Touch (TTP223): TERSENTUH | Hall Effect: idle
```

(c) Serial Monitor saat sensor touch ditekkan

```
Problems  Output  Debug Console  Terminal  Ports
Joystick VRx: 2795 | VRy: 2832 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2791 | VRy: 2831 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2793 | VRy: 2823 | SW: idle | Touch (TTP223): idle | Hall Effect: idle
Joystick VRx: 2797 | VRy: 2832 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2794 | VRy: 2830 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2795 | VRy: 2829 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2800 | VRy: 2830 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2791 | VRy: 2830 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
Joystick VRx: 2797 | VRy: 2832 | SW: idle | Touch (TTP223): idle | Hall Effect: MAGNET TERDETEKSI
```

(d) Serial Monitor saat sensor hall di dekatkan magnet

1.3 Percobaan 2: Sensor Basis Lain (Ultrasonik dan IR Obstacle)

Percobaan 2 membaca sensor ultrasonik HC-SR04 (basis akustik) dan sensor IR obstacle (basis optik). Pin Trig HC-SR04 dihubungkan ke GPIO 5 dan pin Echo ke GPIO 18 melalui pembagi tegangan, yaitu resistor 1 kΩ seri dari Echo ke GPIO dan resistor 2 kΩ dari GPIO ke GND. HC-SR04 mendapat daya dari pin 5V dan GND-nya disatukan dengan GND ESP32. Pin OUT sensor IR obstacle dihubungkan ke GPIO 19, dengan daya dari pin 3.3V.

Dengan Kode Program 2, praktikan menggerakkan sebuah objek mendekat dan menjauh di depan kedua sensor. Jarak dari HC-SR04 dan status sensor IR dibaca di Serial Monitor.

Listing 2: Pembacaan ultrasonik HC-SR04 dan IR obstacle

```
#include <Arduino.h>

```

```
#define TRIG_PIN 5
#define ECHO_PIN 18
#define IR_PIN 19

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(IR_PIN, INPUT);
  digitalWrite(TRIG_PIN, LOW);
}

float readDistanceCM() {
  // Pastikan TRIG LOW
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  // Kirim pulsa 10 us
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Baca durasi pantulan
  long duration = pulseIn(ECHO_PIN, HIGH, 30000); // timeout 30 ms

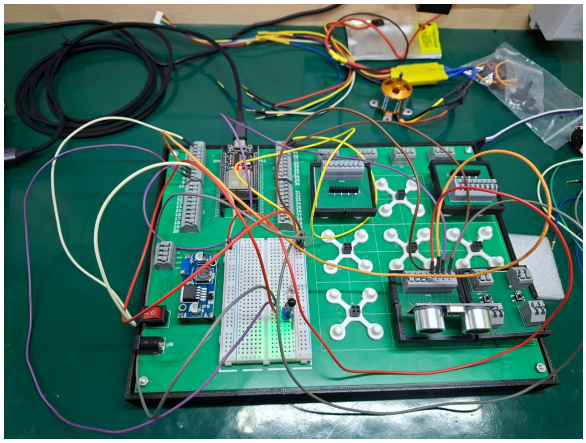
  if (duration == 0)
    return -1.0; // tidak ada objek dalam jangkauan

  // Konversi ke cm
  return duration * 0.0343 / 2.0;
}

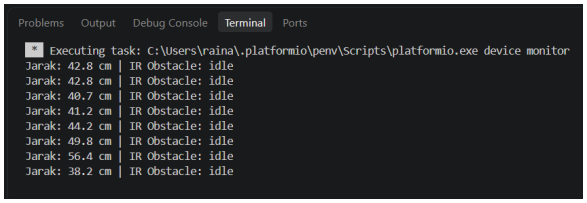
void loop() {
  float distance = readDistanceCM();
  bool obstacle = (digitalRead(IR_PIN) == LOW); // umumnya aktif LOW

  if (distance < 0) {
    Serial.print("Jarak: tidak ada objek");
  } else {
    Serial.printf("Jarak: %.1f cm", distance);
  }
  Serial.printf(" | IR Obstacle: %s\n", obstacle ? "TERDETEKSI" : "idle");
  delay(100);
}
```

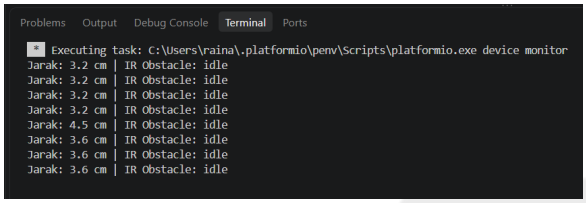
Tabel 2: Hasil Percobaan 2



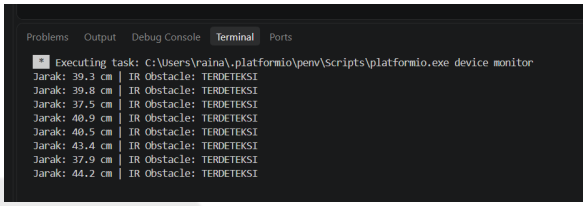
(a) Rangkaian HC-SR04 dan sensor IR obstacle



(b) Serial Monitor saat objek jauh dari sensor



(c) Serial Monitor saat objek dekat sensor Ultrasonik



(d) Serial Monitor saat objek dekat sensor IR

1.4 Percobaan 3: Aktuator Motor DC

Percobaan 3 mengendalikan kecepatan dan arah motor DC JGA25-370 dengan PWM, memakai sumbu X joystick sebagai input. Posisi VRx di sekitar titik tengah (dead zone) membuat motor berhenti, geseran ke satu sisi membuat motor maju, dan geseran ke sisi lain membuat motor mundur, dengan kecepatan sebanding dengan jauhnya geseran. Tombol SW berfungsi sebagai stop darurat. Joystick tetap memakai GPIO 34 untuk VRx dan GPIO 27 untuk SW. Pada driver L298N, jumper ENA dilepas, lalu ENA dihubungkan ke GPIO 25, IN1 ke GPIO 26, dan IN2 ke GPIO 33. Kabel merah motor (M1) dipasang ke OUT1 dan kabel putih (M2) ke OUT2. Terminal +12V L298N mendapat daya dari catu daya eksternal 12V, dan GND catu daya, L298N, serta ESP32 disatukan.

Program untuk percobaan ini ada pada Kode Program 3. Joystick digeser ke kiri dan ke kanan dengan jarak yang berbeda-beda, lalu tombol SW ditekan saat motor sedang berputar. Pada setiap kondisi, praktikan mencatat arah putar, kecepatan motor, dan nilai PWM yang tampil.

Listing 3: Kontrol motor DC via joystick

```
#include <Arduino.h>

//==================== JOYSTICK ====================
#define JOY_VRX 34
#define JOY_SW  27

const int JOY_CENTER = 2048;
const int JOY_DEADZONE = 300; // toleransi di sekitar titik tengah

//==================== MOTOR ====================
```

```
#define ENA 25
#define IN1 26
#define IN2 33

const int pwmChannel = 0;
const int pwmFreq = 1000;
const int pwmResolution = 8;

void motorForward(uint8_t speed) {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  ledcWrite(pwmChannel, speed);
}

void motorBackward(uint8_t speed) {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  ledcWrite(pwmChannel, speed);
}

void motorStop() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  ledcWrite(pwmChannel, 0);
}

void setup() {
  Serial.begin(115200);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(JOY_SW, INPUT_PULLUP);

  ledcSetup(pwmChannel, pwmFreq, pwmResolution); // konfigurasi channel PWM (core 2.x
    )
  ledcAttachPin(ENA, pwmChannel);                // hubungkan channel ke pin ENA

  Serial.println("---------------------------------");
  Serial.println("ESP32 Motor DC via Joystick");
  Serial.println("Geser VRx untuk maju/mundur, tekan SW untuk stop darurat");
  Serial.println("---------------------------------");

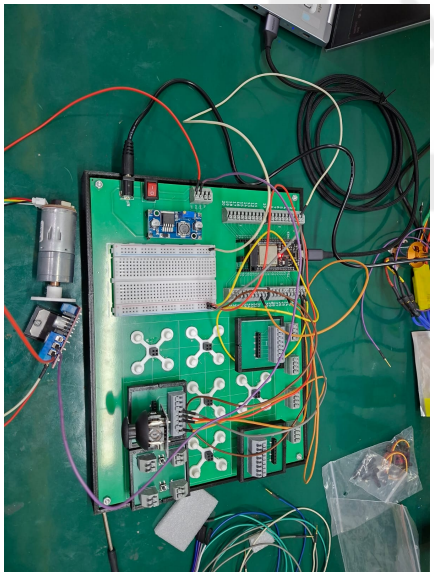
  motorStop();
}

void loop() {
  bool emergencyStop = (digitalRead(JOY_SW) == LOW);

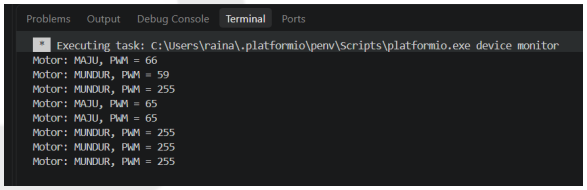
  if (emergencyStop) {
    motorStop();
    Serial.println("STOP DARURAT (SW ditekan)");
    delay(50);
    return;
  }
```

```
64
65    int vrx = analogRead(JOY_VRX);
66    int deviation = vrx - JOY_CENTER;
67
68    if (abs(deviation) < JOY_DEADZONE) {
69      motorStop();
70      Serial.println("Motor: idle (joystick netral)");
71    } else if (deviation > 0) {
72      uint8_t speed = constrain(map(deviation, JOY_DEADZONE, JOY_CENTER, 0, 255), 0,
        255);
73      motorForward(speed);
74      Serial.printf("Motor: MAJU, PWM = %d\n", speed);
75    } else {
76      uint8_t speed = constrain(map(-deviation, JOY_DEADZONE, JOY_CENTER, 0, 255), 0,
        255);
77      motorBackward(speed);
78      Serial.printf("Motor: MUNDUR, PWM = %d\n", speed);
79    }
80
81    delay(100);
82  }
```

Tabel 3: Hasil Percobaan 3



(a) Rangkaian motor DC, L298N, catu daya 12V, dan joystick



(b) Serial Monitor saat motor maju dan mundur

1.5 Percobaan 4: Aktuator Motor Stepper

Percobaan 4 memutar motor stepper 28BYJ-48 sejauh 90° melalui driver ULN2003, berhenti 0,5 detik, lalu mengulang pola tersebut terus-menerus. Pin IN1–IN4 driver ULN2003 dihubungkan ke GPIO 16, 17, 18, dan 19. Motor 28BYJ-48 dicolokkan langsung ke konektor driver. Driver mendapat daya dari catu daya eksternal, dan GND catu daya disatukan dengan GND ESP32.

Poros motor diberi penanda agar sudut putarnya mudah dilihat. Setelah Kode Program 4 di-Upload, praktikan mengamati poros berputar 90°, berhenti, lalu berputar lagi, dan mencocokkannya

dengan pesan di Serial Monitor.

Listing 4: Kontrol motor stepper: putar 90 derajat dengan jeda 0,5 detik

```
#include <Arduino.h>

//==================== STEPPER MOTOR ====================
#define IN1 16
#define IN2 17
#define IN3 18
#define IN4 19

const int motorPins[4] = {IN1, IN2, IN3, IN4};

const int stepSequence[8][4] = {
    {1, 0, 0, 0},
    {1, 1, 0, 0},
    {0, 1, 0, 0},
    {0, 1, 1, 0},
    {0, 0, 1, 0},
    {0, 0, 1, 1},
    {0, 0, 0, 1},
    {1, 0, 0, 1}
};

//==================== KONFIGURASI ====================
const int STEPS_PER_REVOLUTION = 4096;  // 28BYJ-48 dengan gearbox
const int STEPS_PER_90_DEG = STEPS_PER_REVOLUTION / 4;

const int STEP_DELAY_US = 2000;
const int STOP_DELAY_MS = 500;

//==================== MOTOR ====================
void setStep(int step) {
    for (int i = 0; i < 4; i++) {
        digitalWrite(motorPins[i], stepSequence[step][i]);
    }
}

void stopMotor() {
    for (int i = 0; i < 4; i++) {
        digitalWrite(motorPins[i], LOW);
    }
}

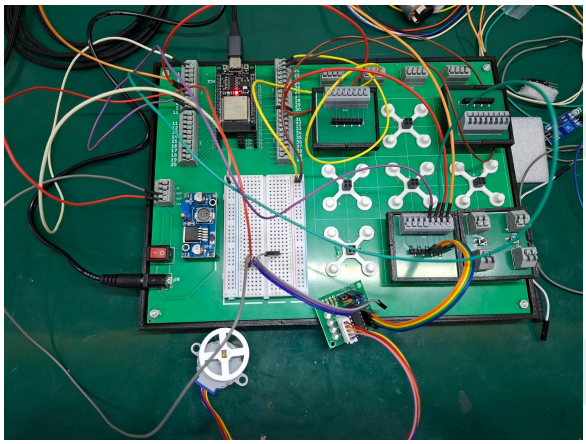
void rotate90Degrees() {
    for (int i = 0; i < STEPS_PER_90_DEG; i++) {
        setStep(i % 8);
        delayMicroseconds(STEP_DELAY_US);
    }
    stopMotor();
}

//==================== SETUP ====================
void setup() {
    Serial.begin(115200);
```

```
    for (int i = 0; i < 4; i++) {
        pinMode(motorPins[i], OUTPUT);
    }
    stopMotor();
    Serial.println("Stepper: 90 degree rotation");
    Serial.println("Stop duration: 500 ms");
}

//==================== LOOP ====================
void loop() {
    Serial.println("Motor berputar 90 derajat");
    rotate90Degrees();
    Serial.println("Motor berhenti 0.5 detik");
    delay(STOP_DELAY_MS);
}
```

Tabel 4: Hasil Percobaan 4



(a) Rangkaian motor stepper 28BYJ-48 dan driver ULN2003



(b) Serial Monitor saat motor berputar 90° dan berhenti

1.6 Percobaan 5: Aktuator Motor Servo

Percobaan 5 menggerakkan servo bolak-balik antara 180° (CW) dan 0° (CCW) memakai library ESP32Servo. Setiap gerakan ditunggu 400 ms sebagai perkiraan waktu gerak, lalu servo berhenti 0,5 detik sebelum bergerak ke arah sebaliknya. File `platformio.ini` memakai pengaturan dasar ditambah `lib_deps = madhephaestus/ESP32Servo@^3.0.0`. Kabel sinyal servo (oranye/kuning) dihubungkan ke GPIO 13, kabel merah ke pin 5V ESP32, dan kabel cokelat ke GND.

Kode Program 5 di-Upload, lalu gerakan lengan servo dicocokkan dengan pesan CW dan CCW di Serial Monitor.

Listing 5: Kontrol motor servo: gerak bolak-balik 0 dan 180 derajat

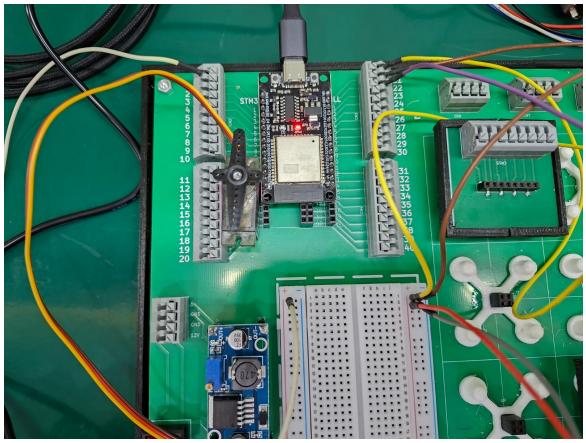
```
#include <Arduino.h>
#include <ESP32Servo.h>

#define SERVO_PIN 13

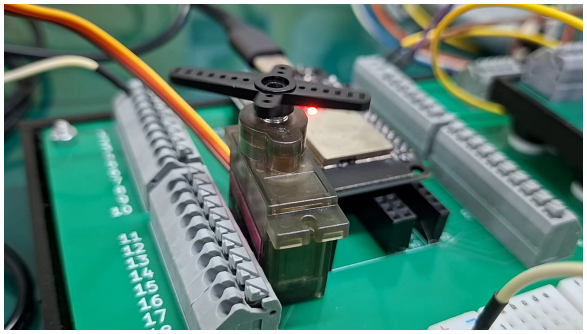
Servo myServo;
```

```
7
8  const int ANGLE_CW = 180;
9  const int ANGLE_CCW = 0;
10
11 const int MOVE_TIME_MS = 400;  // Estimasi waktu gerak servo
12 const int STOP_DELAY_MS = 500; // Jeda setelah gerakan
13
14 void moveServo(int angle, const char* direction) {
15     myServo.write(angle);
16     Serial.printf("Servo bergerak %s ke %d derajat\n", direction, angle);
17
18     // Tunggu estimasi gerakan selesai
19     delay(MOVE_TIME_MS);
20
21     Serial.println("Gerakan selesai (estimasi)");
22     Serial.println("Berhenti selama 0.5 detik");
23
24     // Jeda dimulai setelah waktu gerak
25     delay(STOP_DELAY_MS);
26 }
27
28 void setup() {
29     Serial.begin(115200);
30
31     myServo.setPeriodHertz(50);
32     myServo.attach(SERVO_PIN, 500, 2400);
33
34     myServo.write(ANGLE_CCW);
35     delay(1000);
36
37     Serial.println("MG90S Servo Control Started");
38 }
39
40 void loop() {
41     moveServo(ANGLE_CW, "CW");
42     moveServo(ANGLE_CCW, "CCW");
43 }
```

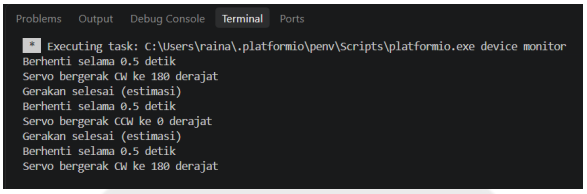
Tabel 5: Hasil Percobaan 5



(a) Rangkaian servo ke ESP32



(b) Posisi lengan servo pada 0° dan 180°



(c) Serial Monitor saat servo bergerak bolak-balik

1.7 Percobaan 6: Aktuator ESC dan Motor Brushless (BLDC)

Percobaan 6 menjalankan motor brushless 2200KV melalui ESC 20A. Sinyal ESC dibangkitkan langsung dengan LEDC tanpa library ESP32Servo. Setelah arming selama 5 detik pada throttle minimum, program menaikkan nilai PWM dari 0 ke 128 secara bertahap, lalu menurunkannya kembali ke 0, dan mengulang pola tersebut. Nilai 0–128 dipetakan ke lebar pulsa 1000–1500 μs agar motor hanya berputar sampai sekitar setengah throttle. File `platformio.ini` memakai pengaturan dasar tanpa baris `monitor_speed`. Kabel sinyal ESC dihubungkan ke GPIO 25 dan kabel GND sinyal ke GND ESP32. Input daya ESC tersambung ke baterai atau catu daya eksternal, dan tiga kabel fasa ESC tersambung ke tiga kabel motor. Propeller tidak dipasang, dan motor dipegang hanya pada bagian kakinya.

Kode Program 6 di-Upload saat switch daya eksternal masih OFF. Kabel USB kemudian dicabut, dan pin VIN serta GND ESP32 dipastikan terpasang ke ESC agar ESP32 mendapat daya dari ESC. Begitu daya eksternal dinyalakan, ESP32 ikut menyala dan mengirim throttle minimum selama 5 detik untuk arming. Praktikan lalu mengamati kecepatan motor saat PWM naik dari 0 ke 128 dan turun kembali ke 0. Jika jeda arming terlewat, daya eksternal dimatikan, tombol EN pada ESP32 ditekan, lalu daya dinyalakan lagi.

Listing 6: Arming ESC dan kontrol kecepatan BLDC naik-turun 0–128 PWM (LEDC)

```
#include <Arduino.h>

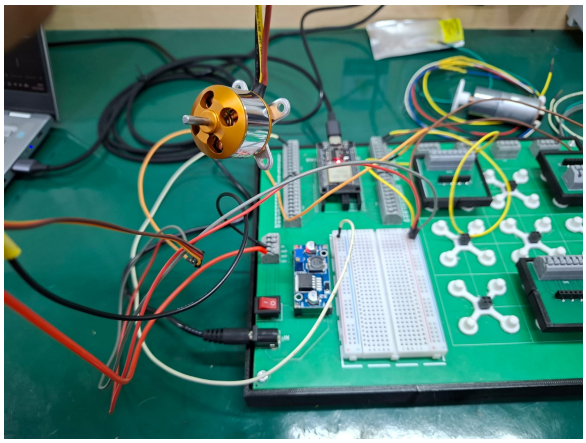
#define ESC_PIN 25
#define ESC_CHANNEL 0

const uint32_t PWM_FREQ = 50;
const uint8_t PWM_RESOLUTION = 16;
```

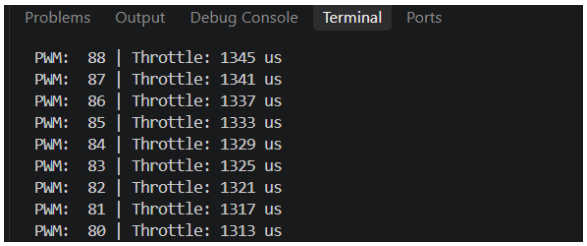
```
8
9  const uint16_t THROTTLE_MIN = 1000; // us (0 PWM)
10 const uint16_t THROTTLE_MAX = 2000; // us (255 PWM)
11 const uint8_t MAX_PWM_TEST = 128;   // Batas uji setengah kecepatan (128 dari 255)
12
13 uint32_t pulseToDuty(uint16_t pulseUs) {
14   return ((uint32_t)pulseUs * 65535UL) / 20000UL;
15 }
16
17 void setThrottlePWM(uint8_t pwmValue) {
18   uint16_t pulseUs = map(pwmValue, 0, 255, THROTTLE_MIN, THROTTLE_MAX);
19   uint32_t duty = pulseToDuty(pulseUs);
20   ledcWrite(ESC_CHANNEL, duty);
21   Serial.printf("PWM: %3d | Throttle: %4d us\n", pwmValue, pulseUs);
22 }
23
24 void setup() {
25   Serial.begin(115200);
26
27   ledcSetup(ESC_CHANNEL, PWM_FREQ, PWM_RESOLUTION);
28   ledcAttachPin(ESC_PIN, ESC_CHANNEL);
29
30   Serial.println("========================");
31   Serial.println("KONTROL ESC & BLDC");
32   Serial.println("========================");
33
34   // Kirim sinyal minimum (PWM 0 = 1000 us) SEBELUM ESC dinyalakan
35   setThrottlePWM(0);
36
37   Serial.println("PWM minimum aktif.");
38   Serial.println("Sekarang nyalakan PSU ESC.");
39
40   // Beri ESC waktu 5 detik untuk arming
41   delay(5000);
42
43   Serial.println("Arming selesai. Motor bergerak naik-turun (0 - 128 PWM).");
44 }
45
46 void loop() {
47   // Naik bertahap dari 0 ke 128 PWM
48   for (int pwm = 0; pwm <= MAX_PWM_TEST; pwm += 1) {
49     setThrottlePWM(pwm);
50     delay(100);
51   }
52
53   delay(500);
54
55   // Turun bertahap dari 128 ke 0 PWM
56   for (int pwm = MAX_PWM_TEST; pwm >= 0; pwm -= 1) {
57     setThrottlePWM(pwm);
58     delay(100);
59   }
60
61   delay(1000);
```

```
62 }
```

Tabel 6: Hasil Percobaan 6



(a) Rangkaian ESP32, ESC, motor BLDC, dan catu daya



(b) Motor BLDC berputar setelah arming

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Sensor Resistif, Kapasitif, dan Induktif

Saat joystick dibiarkan diam, VRx terbaca 2786–2800 dan VRy 2816–2832 (Tabel 1). Titik tengah ini jauh di atas nilai nominal 2048, selisihnya sekitar 750 untuk VRx dan 780 untuk VRy. Dengan rumus konversi linear, nilai 2790 setara dengan sekitar 2,25 V, padahal wiper potensiometer di posisi tengah seharusnya berada di sekitar 1,65 V. Selisih sebesar ini berasal dari toleransi potensiometer, posisi mekanis stick yang tidak benar-benar di tengah, dan respons ADC ESP32 yang tidak linear. Saat stick digeser, nilainya berubah sesuai arah geseran. Pada salah satu posisi, VRy turun sampai 0 dan VRx turun ke 1456–1606. Prinsipnya sama dengan LDR pada Modul 1. Resistansi berubah, tegangan pembagi ikut berubah, lalu ADC membacanya. Pada joystick, resistansi itu berubah karena tangan pengguna menggeser wiper potensiometer, jadi nilainya dipakai sebagai perintah untuk aktuator. Tombol SW terbaca DITEKAN hanya saat stick ditekan ke bawah, karena tombol menarik GPIO 27 ke GND sementara `INPUT_PULLUP` menahannya HIGH di luar kondisi itu.

Modul TTP223 berubah dari idle menjadi TERSENTUH begitu jari menyentuh pad, dan statusnya stabil selama jari tetap menempel. Jari menambah kapasitansi pada elektroda pad, dan IC TTP223 membandingkan perubahan ini dengan ambang internalnya lalu mengeluarkan logika HIGH. Karena itu program cukup membaca pin dengan `digitalRead()` tanpa mengatur ambang sendiri. IC ini mengukur kapasitansi awal pad saat pertama kali diberi daya, jadi pad sebaiknya tidak disentuh ketika ESP32 baru dinyalakan.

Sensor hall effect menampilkan MAGNET TERDETEKSI saat magnet didekatkan. Pada salah satu baris, status sempat kembali idle sebelum terdeteksi lagi. Ini terjadi ketika magnet yang dipegang tangan sedikit bergeser menjauh sehingga medan magnet di sensor turun di bawah ambang. Output modul bersifat aktif LOW, sesuai pembacaan `digitalRead(HALL_PIN) == LOW` di program. Datasheet A3144 menyebut sensor ini bersifat unipolar, yaitu hanya bereaksi pada satu kutub magnet. Jika magnet dibalik, sensor bisa tidak mendeteksi apa pun.

2.2 Percobaan 2: Ultrasonik dan IR Obstacle

HC-SR04 mengukur jarak dari lama pin Echo bernilai HIGH, yaitu waktu yang dibutuhkan gelombang ultrasonik untuk berangkat ke objek dan kembali. Program mengalikan durasi ini dengan kecepatan suara 0,0343 cm/μs lalu membaginya dua karena gelombang menempuh jarak bolak-balik. Saat objek didekatkan, jarak terbaca 3,2–4,5 cm, dan saat objek dijauhkan terbaca 38,2–56,4 cm (Tabel 2). Pembacaan dekat cukup stabil, dengan nilai 3,2 cm berulang empat kali. Pembacaan jauh lebih berfluktuasi, sampai sekitar 18 cm antarbaris. Objek yang dipegang tangan tidak diam, dan pada jarak yang lebih jauh gelombang ultrasonik menyebar lebih lebar sehingga pantulan bisa datang dari benda lain di sekitar objek. Timeout `pulseIn()` sebesar 30 ms setara dengan jarak maksimum sekitar 514 cm, dan jika tidak ada pantulan dalam batas waktu itu, program menampilkan "tidak ada objek". Pembagi tegangan 1 kΩ dan 2 kΩ menurunkan sinyal Echo 5 V menjadi sekitar 3,33 V, di bawah batas input ESP32 sebesar 3,6 V.

Sensor IR obstacle hanya memberi dua keadaan. Output-nya LOW (TERDETEKSI) saat cahaya inframerah dari LED pemancar dipantulkan objek kembali ke fotodioda, dan HIGH (idle) saat tidak ada pantulan. Saat HC-SR04 membaca objek pada 38–56 cm, sensor IR tetap idle. Status IR berubah menjadi TERDETEKSI ketika objek didekatkan ke depan sensor IR, sementara HC-SR04 pada saat yang sama masih membaca 37,5–44,2 cm. Kedua sensor tidak mengarah ke titik yang sama, jadi masing-masing membaca objek yang berbeda di depannya. Jarak deteksi IR diatur oleh potensiometer di modul dan bergantung pada warna objek, karena benda gelap menyerap inframerah. HC-SR04 memberi nilai jarak, sedangkan sensor IR hanya memberi tahu ada atau tidaknya objek di dalam batas jarak tertentu.

2.3 Percobaan 3: Motor DC

Arah putar motor DC ditentukan oleh IN1 dan IN2 pada L298N. Kombinasi IN1 HIGH dan IN2 LOW mengalirkan arus melalui motor ke satu arah, sedangkan kombinasi sebaliknya membalik arah arus. Kecepatan diatur lewat duty cycle PWM 1 kHz pada pin ENA. Fungsi `map()` memetakan deviasi 300–2048 dari titik tengah menjadi PWM 0–255, jadi semakin jauh stick digeser, semakin besar tegangan rata-rata yang diterima motor. Pada pengujian, motor berputar mundur dengan PWM 59 lalu 255 saat stick digeser penuh ke satu sisi, dan berputar maju dengan PWM 65–66 (Tabel 3).

Nilai PWM maju 65 cocok dengan posisi diam joystick yang terukur di Percobaan 1. Program memakai `JOY_CENTER = 2048`, sedangkan VRx saat diam terbaca sekitar 2790. Deviasinya sekitar 745, lebih besar dari dead zone 300, sehingga program menganggap stick digeser ke arah maju. Hasil `map(745, 300, 2048, 0, 255)` adalah 64–65, sama dengan PWM yang tampil di Serial Monitor. Dead zone 300 tidak cukup untuk menutupi pergeseran titik tengah joystick ini, dan motor tetap berputar pelan walaupun joystick tidak disentuh. Masalah ini bisa diatasi dengan membaca VRx beberapa kali di `setup()` saat joystick diam, lalu memakai rata-ratanya sebagai `JOY_CENTER`. Cara lainnya adalah memperbesar dead zone, tetapi rentang kecepatan ke arah maju akan ikut berkurang.

Pada PWM kecil, torsi motor belum cukup untuk melawan gesekan statis pada motor dan gearboxnya. L298N juga menurunkan tegangan sekitar 2 V atau lebih karena transistor di dalamnya, jadi motor tidak menerima 12 V penuh walaupun PWM bernilai 255. Saat SW ditekan, motor langsung berhenti apa pun posisi joystick, karena kondisi stop darurat diperiksa paling awal di `loop()` dan program langsung keluar dengan `return`.

2.4 Percobaan 4: Motor Stepper

Program menjalankan 1024 step dengan pola half-step 8 langkah, dan setiap step berlangsung 2 ms. Satu gerakan 90° memakan waktu sekitar 2,05 detik, atau setara dengan sekitar 7,3 RPM. Serial Monitor menampilkan pesan "Motor berputar 90 derajat" dan "Motor berhenti 0.5 detik" secara bergantian, dan poros motor bergerak sesuai urutan pesan tersebut (Tabel 4). Pesan berputar muncul sebelum gerakan dimulai, karena `Serial.println()` dipanggil sebelum `rotate90Degrees()`. Jika `STEP_DELAY_US` dikurangi, putaran makin cepat. Namun jika nilainya terlalu kecil, rotor tidak sempat mengikuti perpindahan medan magnet, dan motor bergetar atau melewatkan step.

Nilai 4096 berasal dari 64 half-step per putaran rotor dikali rasio gearbox 64:1. Jika rasio gearbox motor yang dipakai tidak tepat 64:1, 1024 step tidak menghasilkan tepat 90°, dan selisih kecil ini menumpuk setiap kali pola diulang. Selama jeda, `stopMotor()` mematikan semua koil, jadi motor tidak menahan posisi secara elektrik dan koil tidak panas. Poros tetap diam karena gearbox reduksi yang besar menahan putaran dari luar.

2.5 Percobaan 5: Motor Servo

Posisi servo ditentukan oleh lebar pulsa. Dengan `attach(SERVO_PIN, 500, 2400)`, sudut 0° dikirim sebagai pulsa 500 μs dan 180° sebagai pulsa 2400 μs pada periode 20 ms. Serial Monitor menampilkan urutan "Servo bergerak CW ke 180 derajat", "Gerakan selesai (estimasi)", "Berhenti selama 0.5 detik", lalu "Servo bergerak CCW ke 0 derajat", dan lengan servo bergerak bolak-balik mengikuti urutan ini (Tabel 5). Rentang 500–2400 μs lebih lebar dari 1–2 ms pada teori, karena servo SG90 atau MG90S butuh pulsa lebih pendek dan lebih panjang dari itu untuk mencapai sudut ujungnya. Rangkaian kontrol di dalam servo membandingkan pulsa tersebut dengan posisi poros yang dibaca potensiometer internal, lalu menggerakkan motor sampai keduanya cocok.

Waktu tunggu 400 ms cukup untuk satu gerakan 180°. Datasheet SG90 menyebutkan kecepatan sekitar 0,1 s per 60° pada 4,8 V, jadi gerakan 180° butuh sekitar 0,3 s. Program tidak mendapat umpan balik dari servo, sehingga pesan "Gerakan selesai (estimasi)" hanya berdasarkan waktu tunggu. Servo mengambil arus cukup besar saat mulai bergerak, dan karena dayanya diambil dari pin 5V ESP32 (dari USB), lonjakan arus ini bisa membuat tegangan turun sesaat. Servo yang lebih besar sebaiknya diberi catu daya terpisah dengan GND yang disatukan.

2.6 Percobaan 6: ESC dan Motor BLDC

ESC hanya mau di-arming jika menerima pulsa 1000 μs yang stabil saat pertama menyala. ESP32 karena itu harus sudah mengirim throttle minimum ketika ESC mulai menyala. Jika ESC menerima throttle di atas minimum saat baru menyala, ESC menolak menjalankan motor. Setelah arming, motor berputar makin cepat ketika PWM naik dan melambat hingga berhenti ketika PWM turun (Tabel 6).

Serial Monitor menunjukkan fase turun, dari PWM 88 dengan throttle 1345 μs sampai PWM 80 dengan throttle 1313 μs. Setiap penurunan satu nilai PWM mengurangi lebar pulsa sekitar 4 μs, sesuai pemetaan 0–255 ke 1000–2000 μs yang memberi $1000/255 \approx 3{,}9\,\mu$s per langkah. Nilai ini juga cocok dengan perhitungan `map()`: PWM 88 menghasilkan $1000 + 88 \times 1000/255 = 1345\,\mu$s. LEDC dikonfigurasi pada 50 Hz dengan resolusi 16-bit. Satu periode 20 ms dibagi menjadi 65.536 nilai, dan setiap nilai mewakili sekitar 0,3 μs, jauh lebih halus dari langkah 3,9 μs yang dibutuhkan. Resolusi 8-bit pada 50 Hz hanya memberi sekitar 78 μs per langkah, terlalu kasar untuk mengatur

throttle dengan halus.

Motor tidak langsung berputar pada PWM kecil karena ESC punya batas throttle minimum sebelum mulai melakukan komutasi. Satu siklus naik-turun berlangsung sekitar 27 detik: 129 langkah naik dan 129 langkah turun dengan jeda 100 ms, ditambah jeda 0,5 detik dan 1 detik. Dengan KV 2200, motor bisa mencapai sekitar 24.000 RPM tanpa beban pada 11,1 V di throttle penuh. Karena itu pengujian dibatasi sampai setengah throttle dan propeller dilepas.

3 Hasil Tugas Modul

3.1 Rancangan Sistem

Tugas modul meminta simulasi jam analog dengan tiga motor stepper pada ESP32, masing-masing untuk jarum milidetik, detik, dan menit. Simulasi dibuat di Wokwi. Setiap motor adalah stepper bipolar 200 step per putaran (1,8° per step) yang dikendalikan driver A4988 dengan sinyal STEP dan DIR, seperti dijelaskan pada Dasar Teori C.7. Satu pulsa STEP memutar motor satu step, dan DIR dibuat HIGH agar semua jarum berputar searah jarum jam.

Kecepatan setiap jarum diatur lewat jarak waktu antar pulsa STEP. Jarum milidetik berputar satu kali per 1000 ms, jadi butuh 200 step per detik atau satu step setiap 5 ms (60 RPM). Jarum detik berputar satu kali per 60 detik, atau satu step setiap 300 ms (1 RPM). Jarum menit berputar satu kali per 60 menit, atau satu step setiap 18 detik (sekitar 0,0167 RPM). Program tidak memakai `delay()`. Setiap jarum punya jadwal step berikutnya sendiri berdasarkan `micros()`, dan jadwal ini ditambah dengan interval tetap setiap kali step dikirim, sehingga kesalahan waktu tidak menumpuk.

Tabel 7: Alokasi pin dan parameter tiap jarum

Jarum	STEP	DIR	Periode putaran	Interval step
Milidetik	GPIO 25	GPIO 26	1 s	5 ms
Detik	GPIO 27	GPIO 14	60 s	300 ms
Menit	GPIO 33	GPIO 32	3600 s	18 s

Pada setiap driver A4988, pin RESET dan SLEEP disambung bersama agar driver aktif, pin ENABLE dibiarkan atau dihubungkan ke GND, dan pin MS1–MS3 dibiarkan LOW untuk mode full-step. Keluaran 1A/1B dan 2A/2B dihubungkan ke koil A dan koil B motor.

3.2 Source Code

Listing 7: Jam analog dengan tiga motor stepper (STEP/DIR)

```
#include <Arduino.h>

const uint32_t STEPS_PER_REV = 200; // 1.8 derajat per step

struct ClockHand {
  const char* name;
  uint8_t stepPin;
  uint8_t dirPin;
  uint32_t stepIntervalUs; // jarak antar step
  uint32_t nextStepUs;     // jadwal step berikutnya
  uint32_t stepCount;      // jumlah step yang sudah dikirim
```

```
};

// Periode satu putaran: 1 s, 60 s, dan 3600 s
ClockHand hands[3] = {
  {"Milidetik", 25, 26, 1000000UL    / STEPS_PER_REV, 0, 0}, //      5 000 us
  {"Detik",     27, 14, 60000000UL   / STEPS_PER_REV, 0, 0}, //    300 000 us
  {"Menit",     33, 32, 3600000000UL / STEPS_PER_REV, 0, 0}, // 18 000 000 us
};

uint32_t startUs = 0;
uint32_t lastPrintMs = 0;

void stepPulse(uint8_t pin) {
  digitalWrite(pin, HIGH);
  delayMicroseconds(2); // A4988 butuh pulsa minimal 1 us
  digitalWrite(pin, LOW);
}

float handAngle(const ClockHand& h) {
  return (h.stepCount % STEPS_PER_REV) * 360.0 / STEPS_PER_REV;
}

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 3; i++) {
    pinMode(hands[i].stepPin, OUTPUT);
    pinMode(hands[i].dirPin, OUTPUT);
    digitalWrite(hands[i].stepPin, LOW);
    digitalWrite(hands[i].dirPin, HIGH); // searah jarum jam
  }
  startUs = micros();
  for (int i = 0; i < 3; i++) {
    hands[i].nextStepUs = startUs + hands[i].stepIntervalUs;
  }
  Serial.println("Jam stepper dimulai");
}

void loop() {
  uint32_t now = micros();

  for (int i = 0; i < 3; i++) {
    // Selisih bertanda tetap benar walaupun micros() overflow
    if ((int32_t)(now - hands[i].nextStepUs) >= 0) {
      stepPulse(hands[i].stepPin);
      hands[i].stepCount++;
      hands[i].nextStepUs += hands[i].stepIntervalUs;
    }
  }

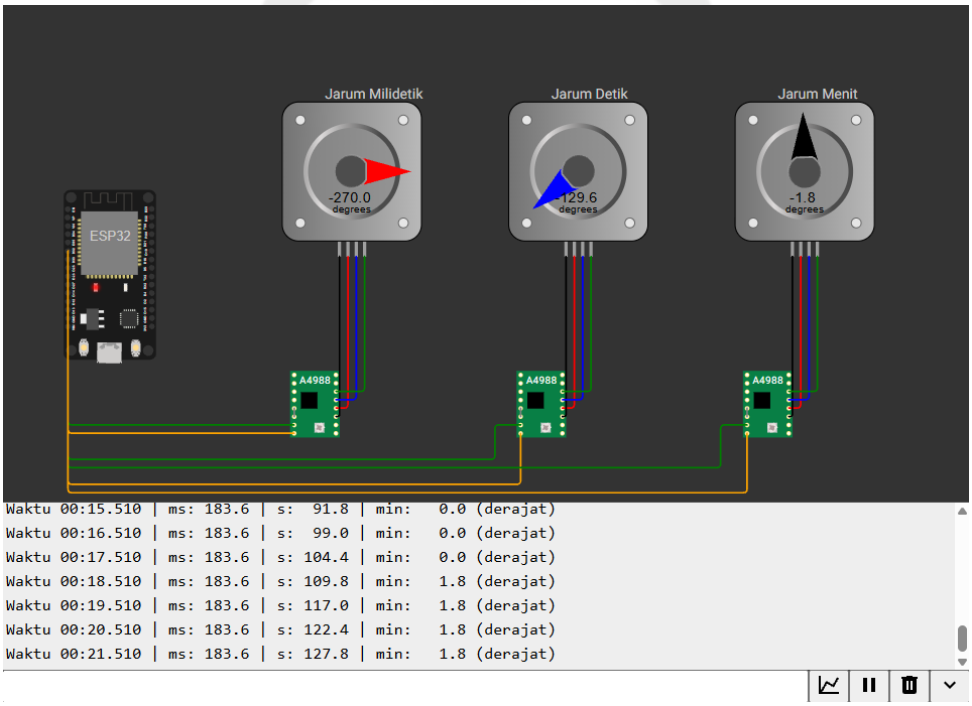
  if (millis() - lastPrintMs >= 1000) {
    lastPrintMs += 1000;
    uint32_t elapsedMs = (micros() - startUs) / 1000;
    Serial.printf("Waktu %02lu:%02lu.%03lu | ms: %5.1f | s: %5.1f | min: %5.1f (
   derajat)\n",
```

```
65                    (elapsedMs / 60000) % 60, (elapsedMs / 1000) % 60, elapsedMs %
      1000,
66                    handAngle(hands[0]), handAngle(hands[1]), handAngle(hands[2]));
67    }
68  }
```

3.3 Penjelasan Program

Setiap jarum disimpan dalam struct `ClockHand` yang berisi pin STEP dan DIR, interval antar step, jadwal step berikutnya, dan jumlah step yang sudah dikirim. Di `loop()`, program membandingkan waktu sekarang dengan jadwal masing-masing jarum. Jika jadwalnya sudah lewat, program mengirim satu pulsa STEP dan memajukan jadwal sebesar satu interval. Jadwal dihitung dari jadwal sebelumnya, bukan dari waktu saat step dikirim, sehingga keterlambatan kecil di satu putaran `loop()` tidak membuat jam makin lama makin terlambat. Perbandingan memakai selisih bertanda `(int32_t)(now - nextStepUs)` agar tetap benar ketika `micros()` kembali ke nol setelah sekitar 71 menit. Setiap detik, Serial Monitor menampilkan waktu berjalan dan sudut ketiga jarum, sehingga posisi motor bisa dicocokkan dengan waktu.

3.4 Hasil Simulasi



Gambar 1: Rangkaian dan serial monitor simulasi jam analog dengan tiga motor stepper dan driver A4988 pada ESP32

3.5 Jawaban Pertanyaan Analisis

1. Bagaimana menentukan RPM dan step tiap motor agar merepresentasikan jarum milidetik, detik, dan menit? Langkah pertama adalah menentukan periode satu putaran tiap jarum. Jarum milidetik berputar satu kali per 1000 ms, jarum detik satu kali per 60 detik, dan jarum menit satu kali per 3600 detik. RPM didapat dari $60/T$ dengan T dalam detik, jadi jarum milidetik berputar 60 RPM,

jarum detik 1 RPM, dan jarum menit $1/60 \approx 0{,}0167$ RPM. Interval antar step didapat dari T dibagi jumlah step per putaran. Dengan motor 200 step per putaran, intervalnya 5 ms untuk jarum milidetik, 300 ms untuk jarum detik, dan 18 detik untuk jarum menit. Setiap step setara dengan 1,8°, jadi jarum detik bergerak 6° per detik melalui sekitar 3,3 step. Jika jarum detik harus berhenti tepat di 60 posisi seperti jam sungguhan, jumlah step per putaran perlu kelipatan 60, misalnya 1200 step per putaran dengan microstepping 1/6, atau jarum dibuat bergerak 6° sekaligus setiap detik.

2. Apakah motor stepper pada simulasi dapat diganti dengan motor servo? Servo standar tidak bisa menggantikan stepper pada jam ini. Servo posisi seperti SG90 hanya bisa bergerak dalam rentang sekitar 0–180°, sedangkan jarum jam harus berputar 360° terus-menerus. Jika dipaksa, jarum hanya bisa menunjukkan setengah lingkaran lalu melompat kembali ke 0°. Servo continuous rotation memang bisa berputar penuh, tetapi sinyalnya mengatur kecepatan, bukan posisi, dan servo tidak memberi umpan balik sudut. Kecepatannya juga berubah mengikuti tegangan dan beban, jadi posisi jarum akan melenceng dan tidak bisa dihitung ulang. Stepper bergerak dengan sudut tetap per step, jadi posisi jarum bisa diketahui hanya dengan menghitung jumlah step yang sudah dikirim.

3. Fungsi driver motor dan alasan setiap motor pada percobaan memakai driver. GPIO ESP32 bekerja pada 3,3 V dengan arus keluaran maksimum sekitar 40 mA per pin. Motor DC JGA25-370 butuh 12 V, stepper 28BYJ-48 butuh 5 V, dan keduanya menarik arus ratusan miliampere. Motor BLDC bahkan bisa menarik arus belasan ampere dari baterai LiPo. Driver menerima sinyal logika kecil dari ESP32 lalu menyalurkan tegangan dan arus besar dari catu daya terpisah ke motor. Driver juga menangani hal yang tidak bisa dilakukan GPIO. L298N memakai H-bridge untuk membalik arah arus motor DC. ULN2003 memakai transistor Darlington untuk menyalakan koil stepper bergantian dan punya dioda yang meredam lonjakan tegangan balik dari koil. ESC melakukan komutasi tiga fasa yang dibutuhkan motor BLDC. Servo pada Percobaan 5 punya driver dan rangkaian kontrol posisi di dalam bodinya, jadi GPIO 13 hanya mengirim sinyal pulsa, sementara daya motornya diambil dari pin 5V.

4 Kesimpulan

Penggunaan berbagai sensor input dan aktuator dalam sistem mikrokontroler mencakup beragam prinsip pengukuran serta metode kendali. Joystick KY-023, TTP223, dan sensor *hall effect* mewakili tiga basis pengukuran yang berbeda; joystick menghasilkan nilai analog 0–4095 dari dua potensiometer, sedangkan TTP223 dan *hall effect* sudah mengolah perubahan kapasitansi serta medan magnet menjadi keluaran digital. Untuk deteksi jarak dan keberadaan objek, HC-SR04 mengukur jarak berdasarkan waktu tempuh pantulan gelombang ultrasonik, sementara sensor *IR obstacle* hanya mendeteksi ada atau tidaknya objek dalam batas jarak yang diatur oleh potensiometernya.

Pada sisi kendali aktuator, kecepatan motor DC dapat diatur melalui *duty cycle* PWM pada pin ENA L298N, sedangkan arah putarannya ditentukan oleh kombinasi logika pin IN1 dan IN2. Penerapan *dead zone* mencegah motor bergerak sendiri saat joystick berada pada posisi netral, dan penekanan tombol SW langsung menghentikan putaran motor. Untuk pergerakan presisi, motor stepper 28BYJ-48 berputar sekitar 90° dengan 1024 *half-step*, di mana jumlah *step* menentukan sudut putar dan jeda antar-*step* menentukan kecepatannya. Sementara itu, posisi motor servo ditentukan oleh lebar

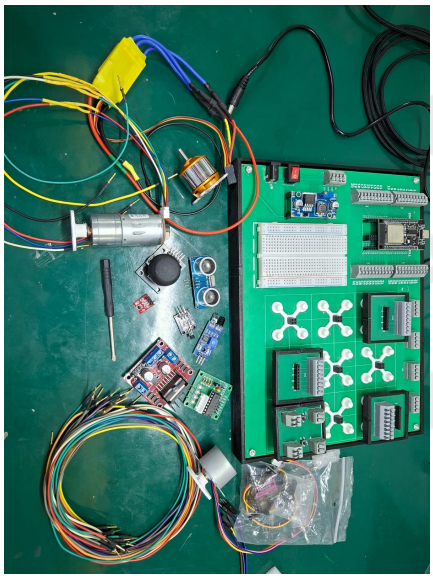
pulsa 500–2400 μs pada frekuensi 50 Hz sehingga servo dapat bergerak bolak-balik antara 0° dan 180° dengan jeda 0,5 detik di setiap ujungnya.

Lebih lanjut, motor BLDC hanya dapat dijalankan setelah ESC melalui proses *arming* menggunakan pulsa 1000 μs, di mana fitur LEDC 16-bit pada 50 Hz mampu membangkitkan sinyal *throttle* 1000–1500 μs secara halus tanpa memerlukan pustaka servo tambahan. Sebagai penerapan pada tugas modul, pengaturan interval *step* sebesar 5 ms, 300 ms, dan 18 detik membuat tiga motor stepper beresolusi 200 *step* per putaran dapat berputar masing-masing satu kali per detik, per menit, dan per jam, menyerupai pergerakan jarum milidetik, detik, dan menit.

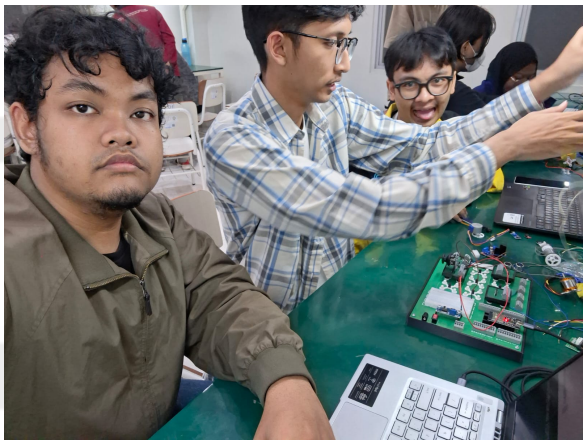
5 Lampiran

5.1 Dokumentasi saat praktikum

Tabel 8: Dokumentasi praktikum



(a) Alat dan bahan praktikum



(b) Praktikan saat melaksanakan praktikum